

I'll explain this React code snippet in detail. This appears to be a component managing an inbox of work items with authentication.

The State Declarations

Token state with lazy initializer:

```
const [token, setToken] = useState(  
  () => (typeof window !== "undefined" ? localStorage.getItem("token") ?? "" : "")  
);
```

This is the most interesting part. The callback function passed to `useState` is a **lazy initializer** — React only calls it once, on the first render, to compute the initial value. Using a function avoids recomputing it on every render. Inside, the `typeof window !== "undefined"` check is a **SSR (server-side rendering) guard**. Since `localStorage` only exists in the browser, this prevents a crash if the code runs on the server (e.g., with Next.js during pre-rendering). The `?? ""` (nullish coalescing) ensures a fallback empty string if the token is `null`.

Typed state with generics:

```
const [inbox, setInbox] = useState<WorkItem[]>([]);  
const [totalItems, setTotalItems] = useState(0);  
const [error, setError] = useState<string | null>(null);  
const [selectedRequestId, setSelectedRequestId] = useState<string | null>(null);
```

These use TypeScript generics. `WorkItem[]` types the inbox as an array of work items. `string | null` allows the value to be either a string or `null` — a common pattern for values that may not exist yet (like an error message or a selected ID).

The `loadInbox` Callback

```
const loadInbox = useCallback(async () => {  
  setLoading(true);  
  setError(null);  
  // ... (rest omitted)  
}, []);
```

`useCallback` memoizes the function so it retains the **same reference** between renders, as long as its dependencies (the empty array `[]`) don't change. This matters because if you pass `loadInbox` as a prop to a child component or use it in a `useEffect` dependency array, a changing reference would trigger unnecessary re-renders or effect re-runs. Since `loadInbox` only calls state setters (which are stable references React guarantees), you can safely use an empty dependency array.

You're very close, but there are a few important corrections. Let me clarify.

The big correction: what `useState` returns

`useState` doesn't create a *single* state variable. It returns **an array of two things**: the current value, and a setter function to change it. That's why the code is written as `const [token, setToken] = useState(...)` — array destructuring.

So when you see:

```
const [totalItems, setTotalItems] = useState(0);
```

You're not calling one function that "returns 0." You're *unpacking* the two items `useState` returns:

- `totalItems` gets the **current value** (which starts as the argument you passed in)
- `setTotalItems` gets the **function that changes** `totalItems`

There's no "function that must be defined afterward" — `useState` *gives* you both of these. They're created for you, automatically, on every render.

The `(null)` and `([])` parts — yes, you've got it

```
useState<WorkItem[]>([]) // initial value = empty array
useState<string | null>(null) // initial value = null
```

The value inside the parentheses is the **initial/default value** that the state holds on the first render. So:

- `useState<WorkItem[]>([]) → inbox starts as [], and inbox is typed as an array of WorkItem`
- `useState<string | null>(null) → error starts as null, and it can be either a string or null`
- `useState<string | null>(null) → selectedRequestId starts as null`

So your interpretation here is correct: the parenthesized value is the default on first run, and the generic (the part in angle brackets) specifies the type.

Fixing your reading of the token line

One correction to how you described it:

"the code fed into `useState(...)` is a function that sets a state variable called `token`"

Not quite. The function you pass to `useState` is the **lazy initializer** — it's just there to *compute the initial value*. It doesn't set anything. It runs once and its return value becomes the starting value of `token`. The thing that *actually changes* `token` is `setToken`, which is returned by `useState`, not the function you pass in.

So the token line reads: "Initialize `token` by reading from `localStorage` (if in the browser), falling back to an empty string. `setToken` is the function I'll use to update it later."

A quick mental model

Think of it like this:

Code	Current value (starts as)	Setter function
<code>useState(0) → [totalItems, setTotalItems]</code>	0	<code>setTotalItems</code>
<code>useState<WorkItem[]>([]) → [inbox, setInbox]</code>	[]	<code>setInbox</code>
<code>useState<string null>(null) → [error, setError]</code>	null	<code>setError</code>
<code>useState<string null>(null) → [selectedRequestId, setSelectedRequestId]</code>	null	<code>setSelectedRequestId</code>
<code>useState(() => ...) → [token, setToken]</code>	result of the lazy initializer	<code>setToken</code>

Later you'll call `setInbox(someArray)` or `setTotalItems(42)` to *update* them — that's the part that sets new values. The `useState` argument only sets the *initial* value.

The async function begins by setting `loading` to `true` (to show a spinner) and clearing any previous `error`, which is the standard "start of fetch" pattern.

A couple of notes worth flagging:

- **The lazy initializer only runs once.** If the token changes in `localStorage` later while the component stays mounted, `token` won't update automatically — you'd need to call `setToken` explicitly (e.g., in a login handler) or re-mount the component.
- **The empty dependency array on `useCallback`** means the callback closes over the initial values. This is fine here since it only uses the setters, but if `loadInbox` were to reference `token` or other changing state, you'd need to add those as dependencies.
- Since the code snippet cuts off mid-function, I'm only describing what's visible. If you'd like, share the rest of `loadInbox` and I can walk through the fetch logic, error handling, and cleanup too.

Great, now I can explain the full picture. Let me walk through the `useCallback`, the fetch logic, and the `useEffect`.

`useCallback` — keeping a stable reference

```
const loadInbox = useCallback(async () => { ... }, []);
```

`useCallback` does two things:

1. **Memoizes the function** — it returns the *same* function reference across renders, as long as the dependencies (the `[]`) don't change.
2. Since the dependency array is **empty**, the function never gets recreated. It's a stable reference for the life of the component.

Why does the stable reference matter here? Look at the `useEffect` below it:

```
useEffect(() => {  
  loadInbox();  
}, [loadInbox]);
```

The effect has `loadInbox` in its dependency array. If `loadInbox` were a **plain** function declared inside the component, it would be recreated *on every render*, meaning its reference changes every render. That would cause the effect to run on every render too — a **loop** of fetching data endlessly. By wrapping it in `useCallback` with `[]`, the reference stays constant, so the effect runs **only once** (on mount). This is the classic reason to combine `useCallback` + `useEffect`.

One subtle catch about the empty array: because `loadInbox` closes over the initial render's values, the `token` it uses is the `token` from the **first render**. If the token changes later (e.g., after login) while this component is already mounted, `loadInbox` would still use the old token. In practice this is often fine for a component that mounts fresh with a logged-in session, but it's worth knowing.

The fetch logic inside

```
setLoading(true);  
setError(null);
```

Standard start-of-fetch: show the spinner, clear the previous error message.

```
const res = await fetch("/api/dashboard/inbox", {  
  method: "GET",  
  headers: {  
    "Content-Type": "application/json",  
    "Authorization": "Bearer " + token,  
  },  
});
```

`fetch` returns a Promise, so `await` pauses until the HTTP request completes. The `Authorization: Bearer <token>` header sends the auth token so the server knows who you are. Note: `Content-Type` isn't strictly needed for a GET with no body, but it's harmless.

```
if (!res.ok) {
  throw new Error("Failed to load inbox.");
}
```

`fetch` **does not throw** on HTTP error statuses (4xx/5xx) — it resolves normally. So you have to check `res.ok` (true when status is 200–299) and handle failure yourself. The `throw` jumps to the `catch` block below. This is a critical detail people often get wrong with `fetch`.

```
const data = await res.json();
setInbox(data.data.items);
setTotalItems(data.data.totalItems);
```

Parses the response body as JSON, then stores the items and total count into state, which triggers a re-render with the new data.

```
catch (err) {
  setError(
    err instanceof Error
      ? err.message
      : "Failed to load inbox."
  );
}
```

Catches **both** the `throw` above and any network failure (server down, no connection — those *do* make `fetch` reject). The `err instanceof Error` check handles the case where the thrown thing isn't a real `Error` object, falling back to a generic message.

```
finally {
  setLoading(false);
}
```

`finally` runs whether it succeeded **or** failed, so the loading spinner always gets turned off. This is the correct place for it — putting it only in `try` or `catch` risks leaving `loading` stuck at `true`.

The `useEffect` — running on mount

```
useEffect(() => {
  loadInbox();
}, [loadInbox]);
```

Effects run **after** the component renders. With `[loadInbox]` as the dependency, it runs:

- once after the first render (mount), and
- again only if `loadInbox`'s reference changes.

Since `loadInbox` is stable (thanks to `useCallback` with `[]`), this effect effectively runs **once**, fetching the inbox as soon as the component appears. This is the idiomatic way to load initial data in React.

Summary of the flow

1. Component mounts → render with defaults (`inbox: [], loading: false`, etc.).
2. `useEffect` fires → calls `loadInbox()`.
3. `loadInbox` sets `loading: true`, sends the GET request with the auth token.
4. On success: updates `inbox` and `totalItems`. On failure: sets `error`.
5. `finally` always sets `loading: false`.
6. New state triggers a re-render, and the UI shows either the data, an error, or a loading indicator.

The `useCallback/useEffect` pairing is what prevents this fetch from running on every single render — which would otherwise hammer your API and cause an infinite feedback loop.

Want me to explain the error-handling pattern (the `err instanceof Error` check) in more depth, or how this would tie into a loading spinner / retry button in the JSX?

This function follows a very similar pattern to `loadInbox`, but with a few important differences worth calling out.

What it does

`handleTransition` sends a **POST** request to transition a workflow item (e.g., approve, reject, forward) to its next state, then refreshes the inbox. It takes two arguments:

- `requestId: string` — which request to act on
- `action: WorkflowAction` — the action to perform (a custom type, likely an enum or union of strings like `"approve"` | `"reject"`)

A `WorkflowAction` type suggests this is a state-machine pattern where each action moves the item through a defined workflow (e.g., `pending` → `approved` → `completed`).

The request

```
const res = await fetch(`/api/daily-requests/${requestId}/transition`, {
  method: "POST",
  headers: {
    "Content-Type": "application/json",
    "Authorization": "Bearer " + token,
  },
  body: JSON.stringify({ action }),
});
```

Notice the differences from `loadInbox`:

- **Template literal URL** — `requestId` is interpolated into the path, so each request targets its own item.
- **method: "POST"** — because this *mutates* state rather than just reading it.
- **body** — a JSON-encoded payload sending the action. Here `Content-Type: application/json` becomes genuinely necessary, since the server needs to know how to parse the body.
- Same auth header as before.

A subtle ordering difference

Compare the two error-handling styles:

```
// In loadInbox:
if (!res.ok) { throw ... } // check status FIRST
const data = await res.json(); // parse AFTER

// In handleTransition:
const data = await res.json(); // parse FIRST
if (!res.ok) { throw ... } // check status AFTER
```

In `handleTransition`, it parses the JSON **before** checking the status. This works if the server always returns JSON — even on errors. In fact, look at line 16: `data?.message` is read from the parsed body. This means the server is expected to include a human-readable `message` field in error responses, and the code surfaces that to the user. The `?.` optional chaining guards against a missing message, falling back to `"Transition failed."`.

This is actually a nicer UX than `loadInbox`'s version — it captures the server's specific error detail rather than a generic message. But it has a risk: **if the server returns an HTML error page (e.g., a 500 from a proxy) instead of JSON, `res.json()` will throw**. That thrown error would be caught by the outer `catch`, so the app won't crash, but you'd lose the specific message.

After success

```
await loadInbox();
```

Once the transition succeeds, it refetches the entire inbox. This keeps the displayed list in sync with what the server now believes — the item you just transitioned should reflect its new state (or disappear if it moved out of the current view, e.g. into "completed"). Because `loadInbox` returns a Promise (it's `async`), `await` ensures the refresh completes before `handleTransition` finishes. This matters if the caller wants to await the whole thing (e.g., to re-enable a button after both operations finish).

The catch block (truncated)

The snippet cuts off at `catch {`, so I can't see the full body, but based on the pattern it almost certainly does something like `setError(...)`. If you'd like, paste the rest of the catch block and I'll continue from there.

One thing to note: `handleTransition` references `token` and `loadInbox`, which are closure values. Unlike `loadInbox` (wrapped in `useCallback` with `[]`), this function is declared as a plain `async` function — so it's **recreated every render**. That's fine here because it's presumably called from event handlers (button clicks), not from a `useEffect` dependency array, so a changing reference doesn't cause problems.

Want me to walk through the rest of the catch block, or discuss how you'd handle disabling buttons while a transition is in flight (e.g., a `transitioning` state)?

Let me untangle a few conflated concepts here, because there are actually three different things mixed up in your questions.

1. "UseEffect sets sth that runs in every render"

This is a misconception. `useEffect` does **not** run on every render — it runs **after** render, but only when its dependencies change. An effect with `[]` runs only once, after the first (mount) render.

The thing that runs on *every* render is the **component body itself** — the code directly inside the function component. That's where `const [token, setToken] = useState(...)`, `const loadInbox = useCallback(...)`, and `const handleTransition = ...` all live. Those execute on every render.

2. "loadInbox is declared as a const — how does it convert into a function?"

It's not a conversion — it was *always* a function. This is just JavaScript syntax where you declare a variable and assign it a function:

```
const loadInbox = useCallback(async () => { ... }, []);
```

Read it as: `useCallback(...)` **evaluates to a function**, and we store that function in a variable named `loadInbox`. The `async () => { ... }` is an arrow function — it's the *function itself*, not a value it returns. So `loadInbox` is a variable whose value is a function. Declaring it with `const` doesn't make it less of a function; it just means you can't reassign `loadInbox` to something else later.

So `loadInbox()` (with parentheses) **calls** that function. `loadInbox` alone (no parens) is just a reference to it.

3. "What is the input of useCallback? The []?"

Exactly — `[]` is the dependency array, which here is empty. You've correctly understood: with `[]`, the memoized function never changes. `useCallback(fn, [])` returns the *same* function object on every render.

Now here's the key insight you're missing:

4. "If it never changes, how can handleSubmit call it — and what's the benefit?"

`loadInbox` being *stored* in a variable means **any other code can call it anytime**, regardless of render cycles. Being "stable" (never changing reference) does **not** prevent it from being called — it only prevents it from being *recreated as a new function object*. Calling is unrelated to rendering.

So:

- A **button's `onClick`** can call `loadInbox()` — e.g., a "Refresh" button.
- `handleTransition` calls `await loadInbox()` after a successful transition (as we saw).
- The `useEffect` calls it once on mount.

All three call the *same* function. That's the whole point.

Now, why use `useCallback` at all in this scenario? The benefit plays out through the `useEffect`:

```
useEffect(() => { loadInbox(); }, [loadInbox]);
```

Consider **what would happen without `useCallback`**:

```
// WITHOUT useCallback
const loadInbox = async () => { ... }; // recreated every render

useEffect(() => { loadInbox(); }, [loadInbox]); // loadInbox changes every render
```

Without `useCallback`, `loadInbox` is a fresh function object on every render. So the effect's dependency `loadInbox` is "different" every render. That means the effect re-runs every render → each render triggers a fetch → the fetch sets state → state change re-renders → effect runs again → **infinite fetch loop**.

With `useCallback(..., [])`, `loadInbox` is the same reference forever, so the effect sees a dependency that never changes → it runs **only once**. The `useCallback` exists specifically to make the `useEffect` run once instead of looping.

That's the real benefit here. It's not about "enabling calls" — it's about **giving the effect a stable dependency so the effect behaves predictably**.

5. The bigger conceptual question: "If anywhere we can call `loadInbox`, what's the benefit of defining it as an effect?"

This is the key distinction to internalize:

- **Calling a function directly** (from a button, or from `handleTransition`) is an *explicit, in-response-to-user-action* thing. You decide when it happens.
- **Putting it in a `useEffect`** is about *reacting to the component's lifecycle/render* — "when the component appears (or when some dependency changes), do this." You're declaring *when* it should happen based on render timing, not on a user action.

The effect on mount handles the **initial data load** automatically — the user didn't click anything, but we still want to fetch the inbox as soon as the page opens. Without the effect, you'd have no trigger for that initial load.

So you're right to notice the overlap: the *function itself* is shared, but the *trigger* differs. The effect provides the "on page load" trigger; buttons/handlers provide the "on user action" trigger. Both run the same `loadInbox`.

Let me know if it'd help to trace through the render/effect lifecycle step by step with a concrete example, or clarify the relationship between the dependency array and when the effect fires.

Here's a concrete, step-by-step trace showing exactly how the mount effect, a refresh button, and `handleTransition` all interact with the single `loadInbox` function.

The component (simplified)

```
function InboxPage() {
  const [token, setToken] = useState("abc123");
  const [inbox, setInbox] = useState([]);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState(null);

  // Same reference every render.
  const loadInbox = useCallback(async () => {
    setLoading(true);
    const res = await fetch("/api/dashboard/inbox", {
      headers: { "Authorization": "Bearer " + token },
    });
    const data = await res.json();
    setInbox(data.data.items);
    setLoading(false);
  }, []);

  // Calls the SAME function, only once, after first mount.
  useEffect(() => {
    loadInbox();
  }, [loadInbox]);

  // Calls the SAME function when a button is clicked.
  const handleRefresh = () => {
    loadInbox();
  };

  return (
    <div>
      {loading && <p>Loading...</p>}
      {error && <p style={{ color: "red" }}>{error}</p>}
      <button onClick={handleRefresh}>Refresh</button>
      <ul>
        {inbox.map(item => <li key={item.id}>{item.title}</li>)}
      </ul>
    </div>
  );
}
```

What happens, step by step

Render #1 (mount).

1. React runs the component body. It calls `useCallback`, which returns function object — call it **F**. `loadInbox` now holds **F**. It also calls `useEffect` with the inner arrow and `[loadInbox]`. The inner arrow captures the same **F**.
2. React renders the JSX: shows "Loading..."? No — `loading` is still `false`, so not yet. Shows the empty `<ul>`.
3. After the render, `useEffect` checks its dependency `[loadInbox]`. It's the first run ever, so the effect fires → calls **F** (i.e., `loadInbox()`).
4. Inside **F**: `setLoading(true)` → next render; then `fetch(...)` begins (async, pauses).

Render #2.

- `loading` is now `true`, so the UI shows "Loading...". The effect dependency `loadInbox` is still **F** — no change → effect does **not** re-run. **No loop.** ✓

Later — fetch resolves.

- `setInbox(data.data.items)` and `setLoading(false)` are called.

Render #3.

- `loading` is `false`, `inbox` has items → the list renders. Effect still sees **F**, no re-run.

User clicks "Refresh" now.

- `handleRefresh` calls `loadInbox()`, which is **F** — the exact same function. Fetch happens again, `loading` toggles, list updates. No effect involved; it was a direct call from the event handler. 

The crucial contrast — what breaks without `useCallback`

Now imagine we remove `useCallback`:

```
const loadInbox = async () => { ... }; // NEW function every render
useEffect(() => { loadInbox(); }, [loadInbox]);
```

Render #1: `loadInbox` is function F1. Effect runs, calls F1.

Render #2 (triggered by `setLoading(true)` from F1): This time the component body creates a **new** function, F2. So `loadInbox` is now F2. The effect's dependency `[loadInbox]` compares old (F1) vs new (F2) → **they differ** → effect re-runs → calls F2.

Render #3 (triggered by F2's `setLoading`): New function F3 → dependency changed again → effect runs again → F3.

F3 sets loading again... which re-renders → F4 → effect runs → F5 → ... **infinite fetch loop**, hammering your API forever. The page never stabilizes (and likely "Loading..." sticks).

So `useCallback` with `[]` exists precisely to **freeze the function reference** so the effect's dependency stays constant and it fires exactly once. Without it, the effect and direct calls would spiral out of control.

The bottom line: `loadInbox` is one

Good question — yes, non-empty dependencies are not just possible, they're common and often the *correct* choice. Let me explain how they work and when you'd use them.

How `useCallback(fn, [token])` works

```
const loadInbox = useCallback(async () => {
  const res = await fetch("/api/dashboard/inbox", {
    headers: { "Authorization": "Bearer " + token }, // <-- uses token
  });
  ...
}, [token]); // <-- depends on token
```

Here's the behavior:

1. **First render:** `useCallback` creates function F1, which *closes over* the current `token` value. It returns F1.
2. **If `token` changes later:** React compares the new dependency value to the old one. They're different, so `useCallback` creates a **new** function F2, closing over the *new* `token`.
3. **If `token` stays the same** between renders (and nothing else in the deps changed): `useCallback` returns the **same** F1 — no new function.

So a non-empty dependency array means: **"recreate this function only when `token` changes; otherwise keep the old one."**

Why this matters — the bug it fixes

Recall the original code used `[]`, which meant `loadInbox` always closed over the **initial** `token` forever. That's fine if the `token` is fixed for the component's life. But consider this scenario:

```
// BAD: token captured once, never updates
const loadInbox = useCallback(async () => {
```

```

    fetch(url, { headers: { Authorization: "Bearer " + token } });
  }, []); // token always = initial value

useEffect(() => { loadInbox(); }, [loadInbox]); // runs once

// later, user logs in with a DIFFERENT account
// → token changes, but loadInbox still uses the OLD token!

```

Here, `loadInbox` is stuck with the original token. If the user re-authenticates (token changes), the fetch still sends the wrong credentials.

```

// GOOD: function is recreated when token changes
const loadInbox = useCallback(async () => {
  fetch(url, { headers: { Authorization: "Bearer " + token } });
}, [token]);

useEffect(() => { loadInbox(); }, [loadInbox]);

```

Now when `token` changes, `loadInbox` gets recreated with the new token, the effect sees a changed dependency, re-runs, and fetches with fresh credentials. **This is the cleanest use case for a non-empty dependency.**

When non-empty dependencies shine

1. **Auth / session changes** — the exact scenario above. You want fetches to follow whatever token/session is current.
2. **Callbacks that change behavior based on state.** E.g., a function that filters data by a `searchQuery`. When the query changes, you want a fresh function that captures it:

```

const onSearch = useCallback((item) => {
  return item.name.toLowerCase().includes(searchQuery.toLowerCase());
}, [searchQuery]);

```

3. **Passing callbacks to memoized children.** If a child component is wrapped in `React.memo`, it only re-renders when its props change. A stable callback (via `useCallback`) prevents unnecessary child re-renders — but only if the callback *shouldn't* change. If it legitimately depends on changing state, you list that state so the child updates correctly.

A rule of thumb

Your dependency array should list **any external value the callback reads**. If you read `token` inside the function, `token` belongs in the array. This "dependency tracking" is what keeps the callback in sync with reality.

The empty `[]` is really just a special case — "this function reads nothing that changes." Most of the time, real callbacks read changing values and need those in the array. The `[]` in your inbox example works *only because* you treat the token as fixed for the page's lifetime.

Danger to avoid: leaving a dependency *out* when you read it leads to stale-closure bugs (the "old value forever" problem). Listing *too many* or reading values *before* defining the function can cause other issues, but leaving out a read value is the most common mistake. That's why React's linter (`react-hooks/exhaustive-deps`) warns you: *"React Hook useCallback has a missing dependency: 'token'."* — it's telling you exactly this.

So: non-empty dependencies are how you make callbacks

Let me address your three points one at a time.

1. "This just considers the token value, not a change-notification mechanism"

You're right — and that's a deliberate design choice, not a flaw. React does **not** use any notification/signal system for dependencies. It's purely **value-based comparison** on each render. When the component renders, React compares the current dependency value to the one from the previous render. If they differ (by reference for objects/arrays, by value for primitives), it recreates the callback.

So there's no event, no subscription, no "token changed" callback. It's reactive in the sense that *a re-render is what triggers the comparison*. In fact, the whole chain works only because setting state (`setToken`) causes a re-render, and that re-render is what lets `useCallback` notice the new value. The word "change" you asked about (point 3) is fundamentally defined as: "a re-render happened, and the dependency value is different from last render."

2. "Is 'mount' the right term for the first render?"

Yes — "mount" is the standard, correct term. In React's lifecycle, a component **mounts** when it's created and inserted into the DOM. That first render is the mount render. The effect with `[]` runs once *after* mount (not during it), which is why people say "runs on mount." Subsequent renders are called **re-renders** (or updates), and removal is **unmount**. So: mount = first render, update = later renders, unmount = removal.

3. Reviewing your mental model of `useCallback`

"I interpret `useCallback` so that it sets something in a space that can't be touched by recreating/re-rendering the main object. So the callbacked entity lives independent of the main object while it is a part of it. It is just revised when dependencies change."

This is close but has one subtle inaccuracy. Let me refine it.

What's right: The callback has a lifespan that's *decoupled* from individual renders. A normal function declared in the component body is destroyed and recreated on every render; `useCallback` preserves a single instance across renders. So yes — it "outlives" any single render of the component.

The subtle fix: `useCallback` is *not* really storing the function in a separate, untouchable "space." What actually happens:

- React keeps a **hidden storage slot** associated with this component instance (this is where all hooks' state lives).
- On each render, `useCallback` reads the *previous* dependency value from that slot, compares it to the *current* one.
- If unchanged → it returns the *previous* function it stored.
- If changed → it creates a **new** function, *overwrites* the stored one, and returns it.

So it's not "a part of the object but untouchable." It's more accurate to say: **the function is stored in React's internal state for that component, and the value you see (`loadInbox`) is simply whatever was last stored**. On each render, `loadInbox` is just a binding to the current stored function.

"Revised when dependencies change" — correct. That's exactly the mechanism in point 3.

3 (continued). What counts as a "change"?

This is the crux. The comparison is done with `Object.is` (like `===` but slightly more precise). So:

- **Primitives** (string, number, boolean, null, undefined): compared by **value**. "abc" equals "abc", 5 equals 5. So changing token from "abc" to "def" counts as a change; keeping it "abc" does not.
- **Objects/arrays/functions** (non-primitives): compared by **reference**. Two objects are "the same" only if they're literally the same object in memory. `{}` is never equal to a different `{}`, even with identical contents.

This has a big practical consequence. If your dependency is an object that you recreate each render...

```
const filters = { status: "open" }; // NEW object every render
const loadUser = useCallback(() => { ... }, [filters]);
```

...then `useCallback` sees a *different reference* every render, so it recreates the callback every render — **defeating the memoization entirely**. This is the classic "dependency changes every render" pitfall. The fix is to either depend on the primitive values instead (`[filters.status]`) or memoize